

Payer / Payee Extraction

This document describes the `extract_payor_payee` function: what it does, why it works the way it does, and exactly how it decides who is the payer and who is the payee. It's written plainly — like a focused student explaining a small, important piece of code — and is ready to paste into `docs/` or hand to an engineer for implementation review.

1. What this function is for

`extract_payor_payee` receives a parsed, structured dictionary (output from a format parser), an optional `customer_name`, and an optional `amount`.

Its job is to map parsed fields into semantic roles — payer and payee —

and return

None

```
{"payer": ..., "payee": ..., "amount": ...}.
```

Important: it does not parse raw narratives, do fuzzy matching, or resolve aliases. Parsers feed it facts; the alias engine and other systems resolve names later.

2. Inputs and output

Input/Output	Type	Description
Inputs:		
<code>parsed</code>	<code>Dict[str, Any]</code>	Parser output (keys may be strings or small dicts like <code>{"value": ...}</code>).

Input/Output	Type	Description
<code>customer_name</code>	<code>Optional[str]</code>	The account holder name (if available).
<code>amount</code>	<code>Optional[float]</code>	Transaction amount (sign may indicate direction).
Output:		
Output	<pre> None { "payer": ..., "payee": ..., "amount": ... } </pre>	Extracted counterparties

3. Priority rules

The function follows a strict ordered rule-set. Use these rules to reason about behavior or to add a new special-case.

Rule A — Explicit role keys win.

If the parsed dict contains a payer-like key (originator, sender, ordering customer, from account, etc.), that value is treated as the authoritative payer.

If it contains a payee-like key (beneficiary, bn, receiver, creditor, etc.), that value is treated as the authoritative payee.

Rule B — Account tokens are marked.

If an explicit role value looks account-like, wrap it as BANK(...) before returning.

Rule C — Preserve single explicit role.

If **exactly one explicit role is present** (payer or payee), return it and fill the other role with `customer_name` if provided (otherwise `None`). Do not overwrite a single explicit role with later inference.

Rule D — If both explicit roles present, return them.

No further inference or direction logic is applied.

Rule E — Narrow format inference only when no explicit roles.

Only if neither payer nor payee was found do we try tightly scoped, format-specific inference (example: PIX). These inferences are guarded and conservative; they must not run if any explicit role exists.

Rule F — Generic counterparty fallback.

If format-specific inference didn't apply, look for generic counterparty keys (entity, related party, to_account/from_account).

If such a counterparty is found and **amount** is present, infer direction from the sign (negative → customer paid → payee is counterparty; positive → customer received → payer is counterparty).

If **amount** is missing, preserve the counterparty value instead of collapsing roles.

Rule G — Amount-only fallback.

If no roles at all are available and only **amount** exists, infer by sign:

negative amount → customer is payer;

positive → customer is payee.

This is a last-resort heuristic.

Rule H — Final fallback: unknown.

If none of the above yields a role, return **payer=None**, **payee=None**, **amount** as given. Never fabricate customer for both roles.